Foundations Capstone Mini-Project — NairaTrack Expense Tracker

A cumulative project covering everything from Weeks 1–9. Pure JavaScript, runs in Node (Termux) — no frameworks, no HTML, no internet.

Goal

Build **NairaTrack**, a small command-line expense tracker that keeps a list of expenses in memory and produces a spending **report**: total spent, a breakdown per category, your biggest expense, and whether you're within budget.

The program takes no live input (reading user input isn't covered yet). Your data lives in an array at the top of the file, and your **functions** process it and print results — the same pattern a backend uses to load rows and return a summary.

By the end you'll have used variables, operators, control flow, functions, arrays, objects, and the big three (`map`, `filter`, `reduce`) — all in one program — and pushed it to GitHub.

Step 1 — Break it down in plain English (REQUIRED, before any JavaScript)

Do not open the code editor yet. The first 80% of programming is thinking; typing is the last 20% (Week 4). Before writing any JavaScript, write out **how the program works in plain English**. This is graded, not optional.

Create a file called **`PLAN.md`** and fill in these three parts:

A. Understand — what is this program supposed to do? In your own words, describe the job and list every line you expect the final report to print.

B. Decompose — break the big problem into small steps. List the small jobs the program must do (e.g. "add up all the amounts", "group expenses by category", "find the most expensive one"). One job ≈ one function.

C. Pseudocode — write the logic in plain steps for EACH function. Plain English, numbered steps. No JavaScript syntax. Here is the level we expect, using `totalSpent` as a worked example:

> **`totalSpent` — adds up every expense amount**
>
> 1. Start a running total at 0.
> 2. Go through each expense in the list, one by one.
> 3. For each expense, add its amount to the running total.
> 4. After the last expense, give back (return) the running total.

Now write your own pseudocode for **`categoryTotals`** and **`biggestExpense`** the same way (these are the two trickiest — plan them carefully).

> **Why this matters:** Fixing a wrong plan is far faster than fixing wrong code. Commit `PLAN.md` **first**, before any `.js` file.

Finish `PLAN.md` before moving on. While coding (Step 2), follow the rest of **UPER**: **E**xecute one function at a time (run after each), then **R**eflect — read your code back and DRY up any repetition.

Step 2 — Write the code

Create a file `nairatrack.js` and run it with `node nairatrack.js`. Build it **one function at a time, in the order of your plan**, running after each. Start from this skeleton:

```javascript
/**
 * NairaTrack — Foundations Capstone Mini-Project
 * A console expense tracker. Run with: node nairatrack.js
 */

// === DATA (your in-memory "database") ===
// Each expense is an object. This is an "array of objects" — how real data looks.

const expenses = [
    { id: 1, title: "Transport to class", amount: 500,  category: "transport" },
    { id: 2, title: "Lunch",              amount: 800,  category: "food" },
    { id: 3, title: "Data subscription",  amount: 3500, category: "data" },
    { id: 4, title: "Notebook & pen",     amount: 1200, category: "study" },
    { id: 5, title: "Dinner",             amount: 950,  category: "food" },
    { id: 6, title: "Keke home",          amount: 400,  category: "transport" },
];

const MONTHLY_BUDGET = 8000;
```

```
// === HELPERS ===

// Format a number as Naira, e.g. naira(3500) -> "N3,500"
function naira(amount) {
    // TODO: return a template-literal string starting with "N" and the amount.
    // (Stretch: add thousands separators with amount.toLocaleString())
}

// === 1. ADD AN EXPENSE ===

function addExpense(title, amount, category) {
    // TODO: guard the input — if amount is not a number or is <= 0,
    //       print "Invalid amount" and return WITHOUT adding anything.
    // TODO: generate the next id (one higher than the last expense's id, or 1 if
empty).
    // TODO: push a new expense object onto `expenses`.
    // TODO: print: "Added: <title> (<naira(amount)>)"
}

// === 2. LIST EVERYTHING ===

function listExpenses() {
    // TODO: print a header "--- All Expenses ---"
    // TODO: loop over `expenses` and print each as:
    //       "#<id>  <title> — <naira(amount)>  [<category>]"
}

// === 3. TOTAL SPENT  (use reduce) ===

function totalSpent() {
    // TODO: use reduce to add up every expense.amount and RETURN the number.
}

// === 4. EXPENSES IN ONE CATEGORY  (use filter) ===

function byCategory(category) {
    // TODO: use filter to return only the expenses whose category matches
    //       (make the match case-insensitive).
}

// === 5. TOTAL PER CATEGORY ===

function categoryTotals() {
    // TODO: build and RETURN an object like { transport: 900, food: 1750, ... }
    //       (loop over expenses, or use reduce with an object accumulator).
}
```

```
// === 6. BIGGEST EXPENSE ===

function biggestExpense() {
    // TODO: find and RETURN the single expense object with the highest amount.
    //       (a loop that tracks the max, or reduce.)
}

// === 7. BUDGET CHECK ===

function budgetCheck() {
    const spent = totalSpent();
    // TODO: compare `spent` to MONTHLY_BUDGET.
    //       If within budget, print "Within budget — <naira(remaining)> left".
    //       If over,       print "OVER budget by <naira(overBy)>!".
    //       (Use if/else, or a ternary.)
}

// === 8. THE REPORT — ties it all together ===

function report() {
    // TODO: print a summary that calls the functions above:
    //   - a title line
    //   - listExpenses()
    //   - "Total spent: <naira(totalSpent())>"
    //   - the per-category breakdown from categoryTotals()
    //   - "Biggest expense: <title> (<naira(amount)>)"
    //   - budgetCheck()
}

// === RUN IT ===
report();
```

Core requirements (must pass — 70%)

Your program must:

0. **`PLAN.md`** exists and was written **first** — a plain-English breakdown (understand + decomposition + pseudocode) of the solution, committed before any JavaScript.
1. **`naira()`** formats an amount as a Naira string using a **template literal**.
2. **`addExpense()`** **validates** the amount (rejects non-numbers and values ≤ 0) before adding — use a comparison/guard and an early `return`.
3. **`listExpenses()`** prints every expense using a **loop**.
4. **`totalSpent()`** uses **`reduce`** and **returns** the number (it returns, it doesn't just log).

5. `byCategory()` uses `filter` and returns the matching array.
6. `categoryTotals()` returns an **object** of totals per category.
7. `biggestExpense()` returns the correct expense **object**.
8. `budgetCheck()` uses **if/else or a ternary** to report remaining or over-budget.
9. `report()` runs and prints a clean, readable summary.
10. The code is committed to **Git** and pushed to a **GitHub** repository.

Sample of the kind of output expected

```
None
===== NairaTrack Report =====
--- All Expenses ---
#1  Transport to class — N500  [transport]
#2  Lunch — N800  [food]
...
Total spent: N7,350
By category:
  transport: N900
  food: N1,750
  data: N3,500
  study: N1,200
Biggest expense: Data subscription (N3,500)
Within budget — N650 left
```

Your exact numbers and formatting can differ — match the *shape*.

Stretch goals (optional — for when the core works)

- **Sort** the expense list from highest to lowest amount before printing (`sort`).
- **Average expense**: add `averageExpense()` (total ÷ count) and print it.
- **Needs vs wants**: tag each expense and report what % of spending was "needs" — uses a **higher-order function**.
- **Export**: print the whole report as a single JSON string with `JSON.stringify(report, null, 2)` — this is the format that travels over the network (Week 7).
- Call `addExpense()` a few times at the bottom (including one **invalid** amount) to prove your validation works, then re-run `report()`.

What you must NOT do

- Don't repeat the same logic — extract it into a function (**DRY**).
- Don't write the whole program as one block — use the named functions above.
- No libraries, no `npm install` — plain JavaScript only.

Grading rubric (100 points)

Area	**Points**	**What we look for**
Plain-English plan (`PLAN.md`)	**15**	A real breakdown done **before** the code: understand + decomposition + numbered pseudocode for at least `totalSpent`, `categoryTotals`, and `biggestExpense`. No JavaScript syntax in the plan.
It runs	10	`node nairatrack.js` runs with no errors and prints a report.
Functions & return values	18	Logic is split into the named functions; functions that should `return` a value do (not just `console.log`).
`map` / `filter` / `reduce`	18	`reduce` for the total, `filter` for category, used correctly (not replaced by manual loops everywhere).
Arrays & objects	13	Correctly reads/builds arrays of objects and the category-totals object.
Control flow & validation	13	Loops, conditionals, and the `addExpense` input guard all work.

Area	Points	What we look for
Clean code	5	Good names, `const`/`let` used sensibly, a few helpful comments.
Git & GitHub	8	`PLAN.md` is the **first commit**, then **at least 3 meaningful commits** total, plus a short `README.md` explaining how to run it.

Pass mark: 70/100. Your `PLAN.md` and code should match — a function that does something the plan never mentioned signals the planning step was skipped.